


PGC306B: Tópicos Especiais em IA

Aprendizado de Máquina e Sistemas de Aprendizado Bio-inspirados

Supervised Learning: Neural Networks

Prof. Dr. Murillo Guimarães Carneiro
mgcarneiro@ufu.br

Faculdade de Computação
Universidade Federal de Uberlândia

Atualizado em 09/2019

Objectives

- Present basic concepts about neural networks;
- Present Perceptron and Multi-layer Perceptron networks;
- Discuss about hypothesis space search and inductive bias of MLP.

Outline

Introduction

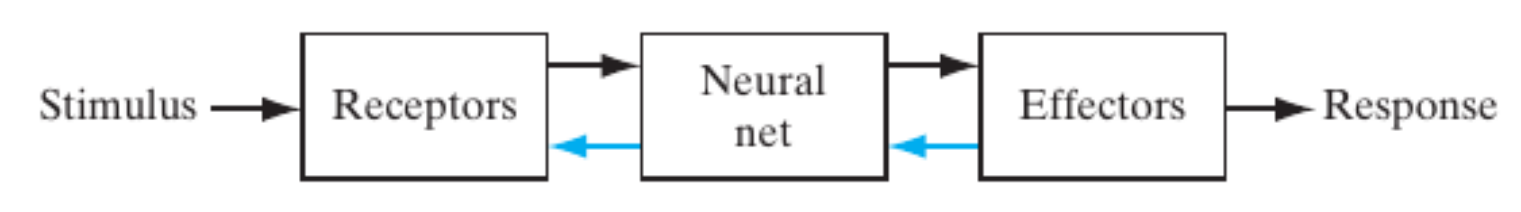
Neuron

Perceptron

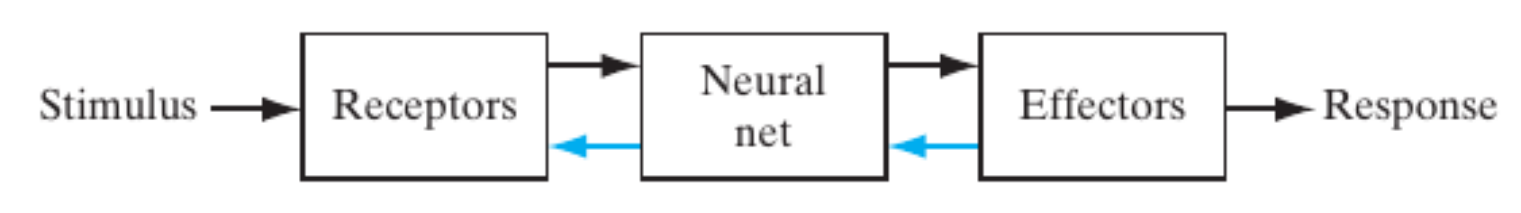
Multi-layer Perceptron

Inductive Bias and Final Remarks

The human brain inspiration

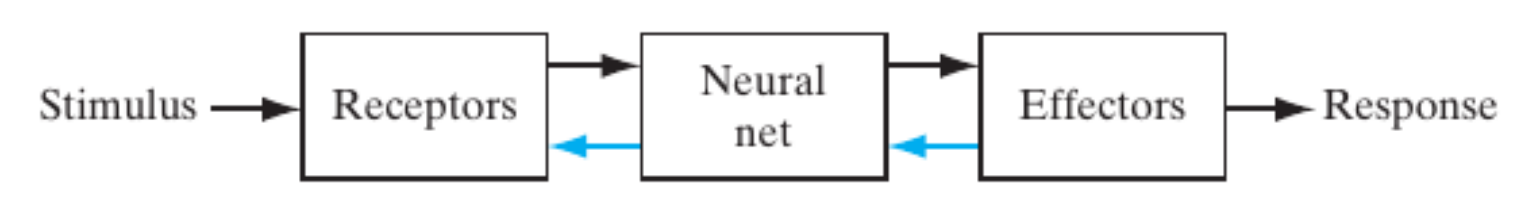


The human brain inspiration



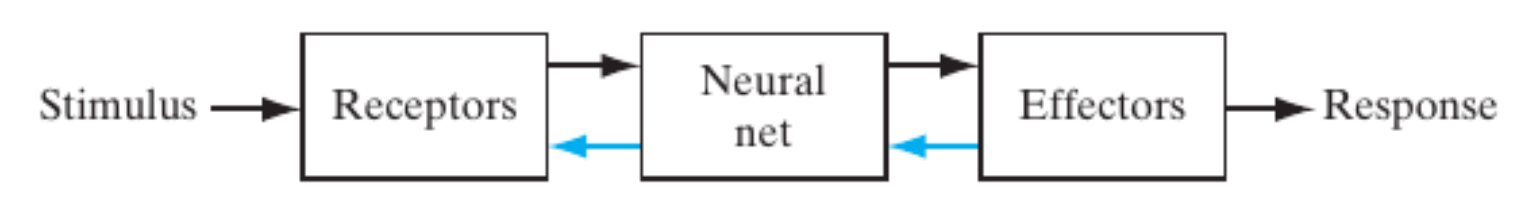
- Neuron switching time $\approx$ .001 second

The human brain inspiration



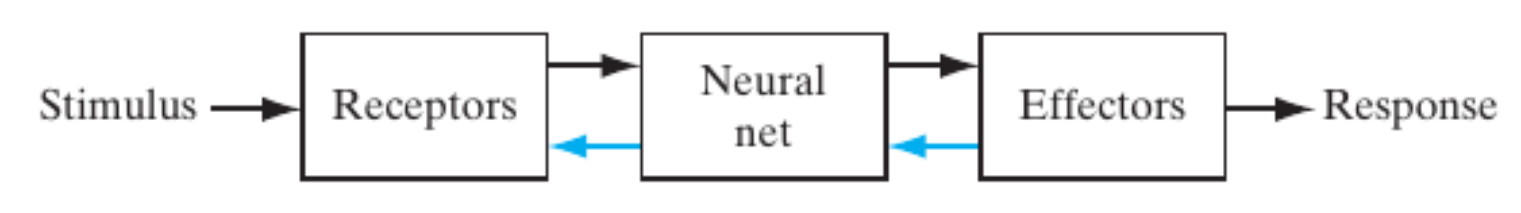
- Neuron switching time $\approx$ .001 second
- Number of neurons $\approx 10^{10-11}$

The human brain inspiration



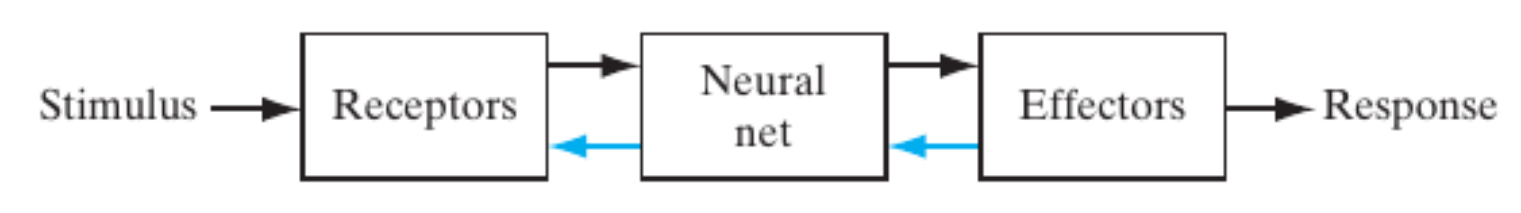
- Neuron switching time $\approx$ .001 second
- Number of neurons $\approx 10^{10-11}$
- Connections per neuron $\approx 10^{4-5}$

The human brain inspiration



- Neuron switching time $\approx$ .001 second
- Number of neurons $\approx 10^{10-11}$
- Connections per neuron $\approx 10^{4-5}$
- Scene recognition time $\approx$ .1 second

The human brain inspiration



- Neuron switching time $\approx$ .001 second
- Number of neurons $\approx 10^{10-11}$
- Connections per neuron $\approx 10^{4-5}$
- Scene recognition time $\approx$ .1 second
- 100 inference steps doesn't seem like enough

$\rightarrow$ much parallel computation

Human brain vs machine

- Neurons are 5-6 orders of magnitude slower than silicon logic gates

Human brain vs machine

- Neurons are 5-6 orders of magnitude slower than silicon logic gates
  - Silicon chip *rightarrow* $\approx$ nanosecond range
  - Neural events *rightarrow* $\approx$ milisecond range

Human brain vs machine

- Neurons are 5-6 orders of magnitude slower than silicon logic gates
  - Silicon chip *rightarrow* $\approx$ nanosecond range
  - Neural events *rightarrow* $\approx$ milisecond range
- However, human brain compreheends an enormously efficent structure
  - $\approx$10 billions of neurons and $\approx$1000 trillions of connections

Human brain vs machine

- Neurons are 5-6 orders of magnitude slower than silicon logic gates
  - Silicon chip *rightarrow* $\approx$ nanosecond range
  - Neural events *rightarrow* $\approx$ milisecond range
- However, human brain compreheends an enormously efficent structure
  - $\approx$10 billions of neurons and $\approx$1000 trillions of connections
- It is also a very efficient energetic structure
  - 10^{-16} joules per operation per second

Human brain vs machine

- Neurons are 5-6 orders of magnitude slower than silicon logic gates
  - Silicon chip *rightarrow* $\approx$ nanosecond range
  - Neural events *rightarrow* $\approx$ milisecond range
- However, human brain compreheends an enormously efficent structure
  - $\approx$10 billions of neurons and $\approx$1000 trillions of connections
- It is also a very efficient energetic structure
  - 10^{-16} joules per operation per second
  - Best computers are orders of magnitude larger ($\approx 10^{-6}$)

Properties of neural nets

- Many neuron-like threshold switching units

Properties of neural nets

- Many neuron-like threshold switching units
- Many weighted interconnections among units

Properties of neural nets

- Many neuron-like threshold switching units
- Many weighted interconnections among units
- Highly parallel, distributed process

Properties of neural nets

- Many neuron-like threshold switching units
- Many weighted interconnections among units
- Highly parallel, distributed process
- Emphasis on tuning weights automatically

When to consider neural networks?

- Input is high-dimensional discrete or real-valued (e.g. raw sensor input)

When to consider neural networks?

- Input is high-dimensional discrete or real-valued (e.g. raw sensor input)
- Output is discrete or real valued

When to consider neural networks?

- Input is high-dimensional discrete or real-valued (e.g. raw sensor input)
- Output is discrete or real valued
- Output is a vector of values

When to consider neural networks?

- Input is high-dimensional discrete or real-valued (e.g. raw sensor input)
- Output is discrete or real valued
- Output is a vector of values
- Possibly noisy data

When to consider neural networks?

- Input is high-dimensional discrete or real-valued (e.g. raw sensor input)
- Output is discrete or real valued
- Output is a vector of values
- Possibly noisy data
- Form of target function is unknown

When to consider neural networks?

- Input is high-dimensional discrete or real-valued (e.g. raw sensor input)
- Output is discrete or real valued
- Output is a vector of values
- Possibly noisy data
- Form of target function is unknown
- Human readability of result is unimportant

Outline

Introduction

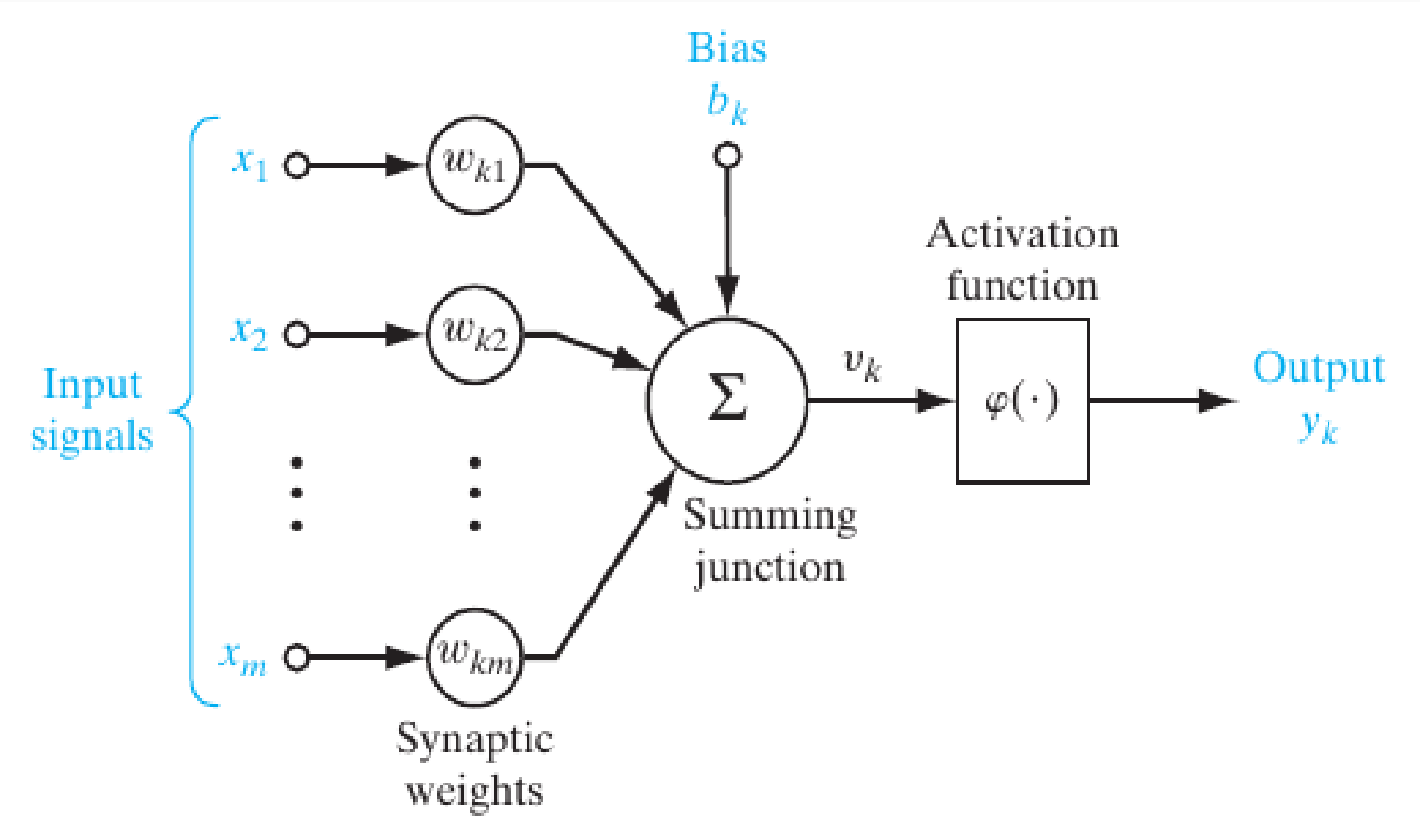
Neuron

Perceptron

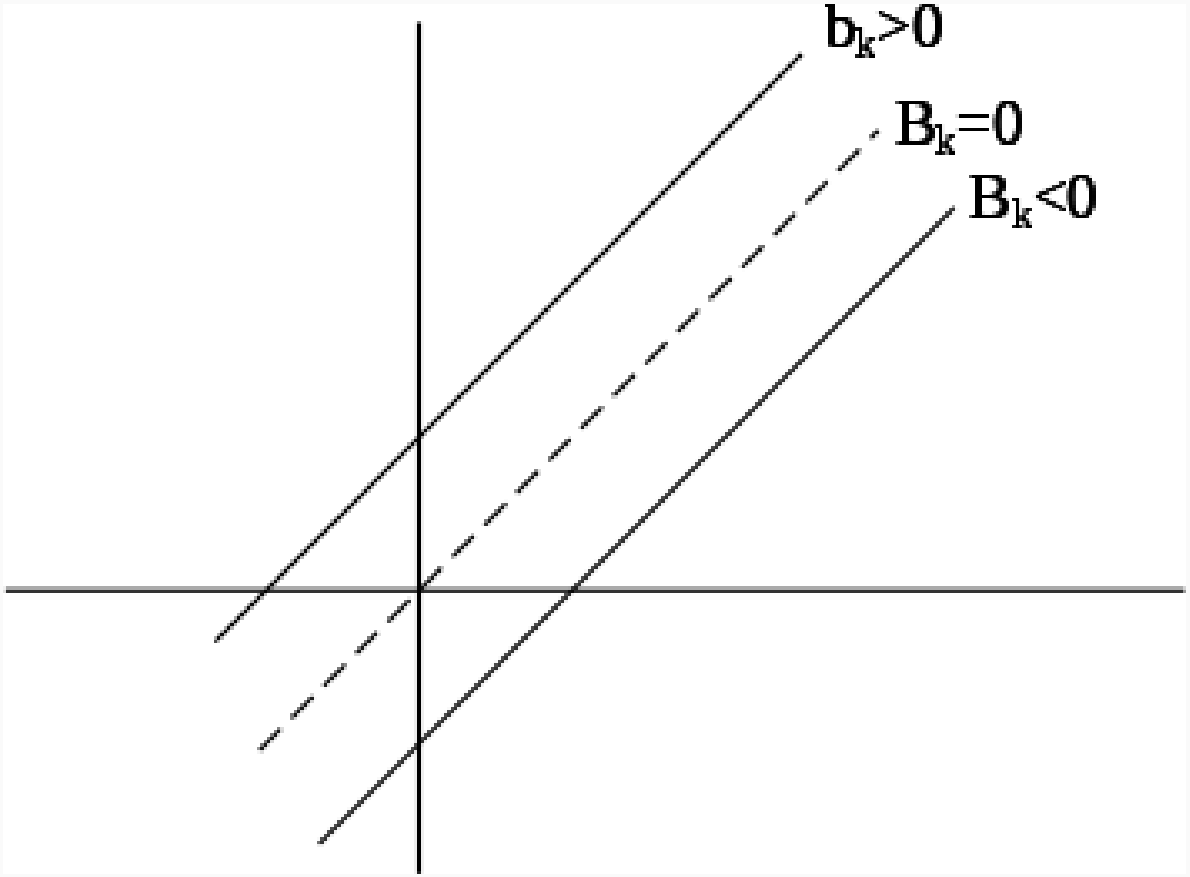
Multi-layer Perceptron

Inductive Bias and Final Remarks

McCulloch-Pitts model



The bias



Neuron

Activation function

- Threshold function (hard limiter): $f(x) = \begin{cases} 1 & \text{if } x \geq 0 \\ 0 & \text{otherwise.} \end{cases}$

Activation function

- Threshold function (hard limiter): $f(x) = \begin{cases} 1 & \text{if } x \geq 0 \\ 0 & \text{otherwise.} \end{cases}$
- Piecewise-Linear function: $f(x) = \begin{cases} 1 & x \geq 1/2 \\ x & -1/2 < x < 1/2 \\ 0 & v \leq -1/2. \end{cases}$

Activation function

- Threshold function (hard limiter): $f(x) = \begin{cases} 1 & \text{if } x \geq 0 \\ 0 & \text{otherwise.} \end{cases}$
- Piecewise-Linear function: $f(x) = \begin{cases} 1 & x \geq 1/2 \\ x & -1/2 < x < 1/2 \\ 0 & v \leq -1/2. \end{cases}$
- Sigmoid function: $f(x) = \frac{1}{1+e^{(-x)}}$

Outline

Introduction

Neuron

Perceptron

Multi-layer Perceptron

Inductive Bias and Final Remarks

Perceptron

The basics

- Created by Rosenblatt (1958)

Perceptron

The basics

- Created by Rosenblatt (1958)
- The first model for learning with a teacher (supervised learning)

Perceptron

The basics

- Created by Rosenblatt (1958)
- The first model for learning with a teacher (supervised learning)
- The simplest form of a neural network used for the classification of patterns said to be linearly separable

Perceptron

The basics

- Created by Rosenblatt (1958)
- The first model for learning with a teacher (supervised learning)
- The simplest form of a neural network used for the classification of patterns said to be linearly separable
- It consists of a single neuron with adjustable weights and bias

Perceptron

The basics

- Created by Rosenblatt (1958)
- The first model for learning with a teacher (supervised learning)
- The simplest form of a neural network used for the classification of patterns said to be linearly separable
- It consists of a single neuron with adjustable weights and bias
- The perceptron built around a single neuron is limited to performing pattern classification with only two classes

The basics

- Created by Rosenblatt (1958)
- The first model for learning with a teacher (supervised learning)
- The simplest form of a neural network used for the classification of patterns said to be linearly separable
- It consists of a single neuron with adjustable weights and bias
- The perceptron built around a single neuron is limited to performing pattern classification with only two classes
- With more than one neuron, it may perform classification with more than two classes (must be linearly separable too).

Processing

- The Perceptron output can be obtained this way:

$$o(x_1, \ldots, x_n) = \begin{cases} 1 & \text{if } w_0 + w_1 x_1 + \cdots + w_n x_n > 0 \\ 0 & \text{otherwise.} \end{cases}$$

Processing

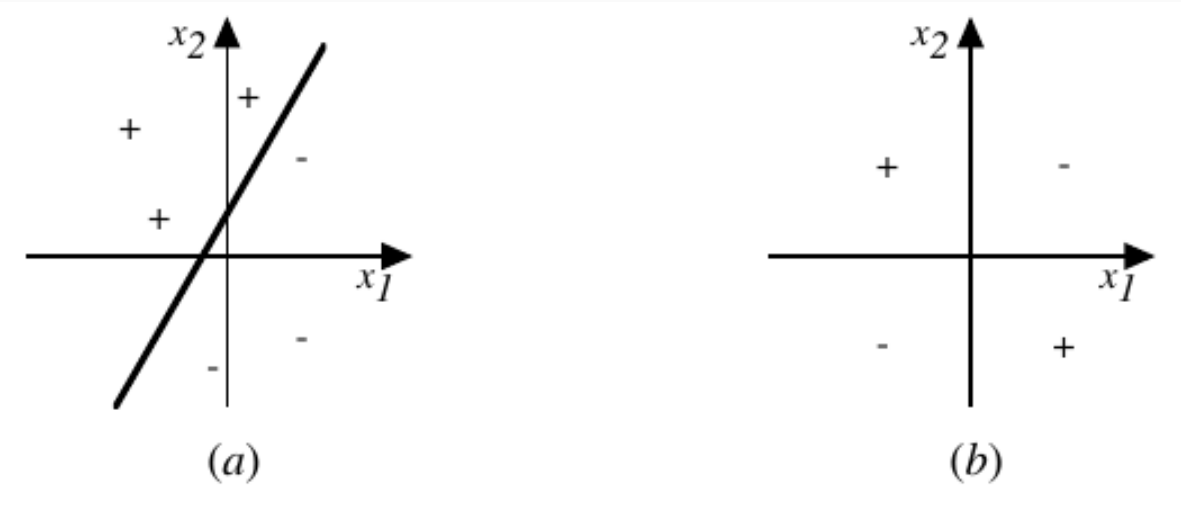
- The Perceptron output can be obtained this way:

$$o(x_1, \ldots, x_n) = \begin{cases} 1 & \text{if } w_0 + w_1 x_1 + \cdots + w_n x_n > 0 \\ 0 & \text{otherwise.} \end{cases}$$

- Sometimes we'll use simpler vector notation:

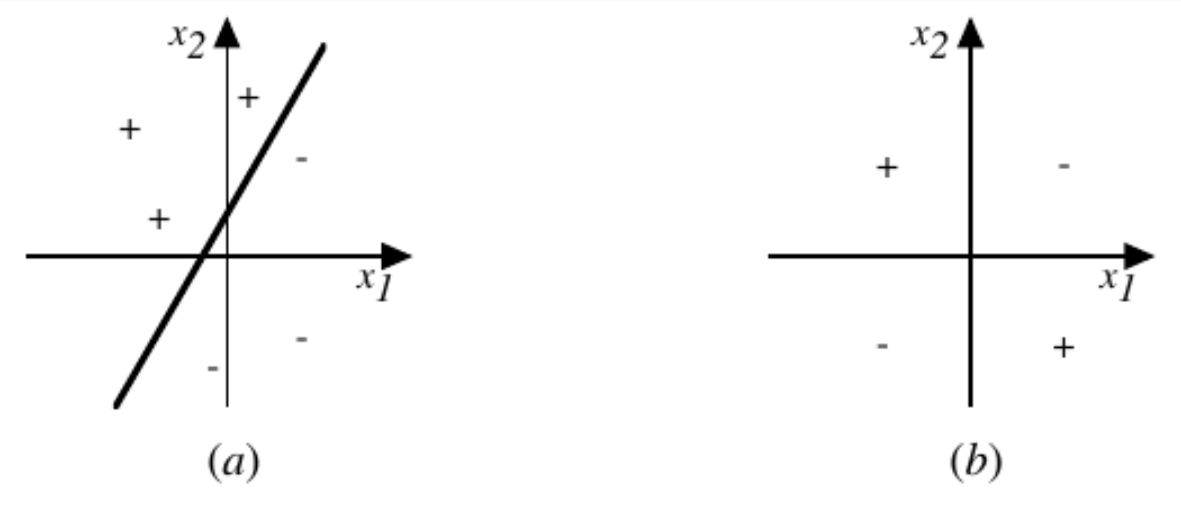
$$o(\vec{x}) = \begin{cases} 1 & \text{if } \vec{w} \cdot \vec{x} > 0 \\ 0 & \text{otherwise.} \end{cases}$$

Representativeness



- Represents some useful functions:
- What weights represent $g(x_1, x_2) = AND(x_1, x_2)$?

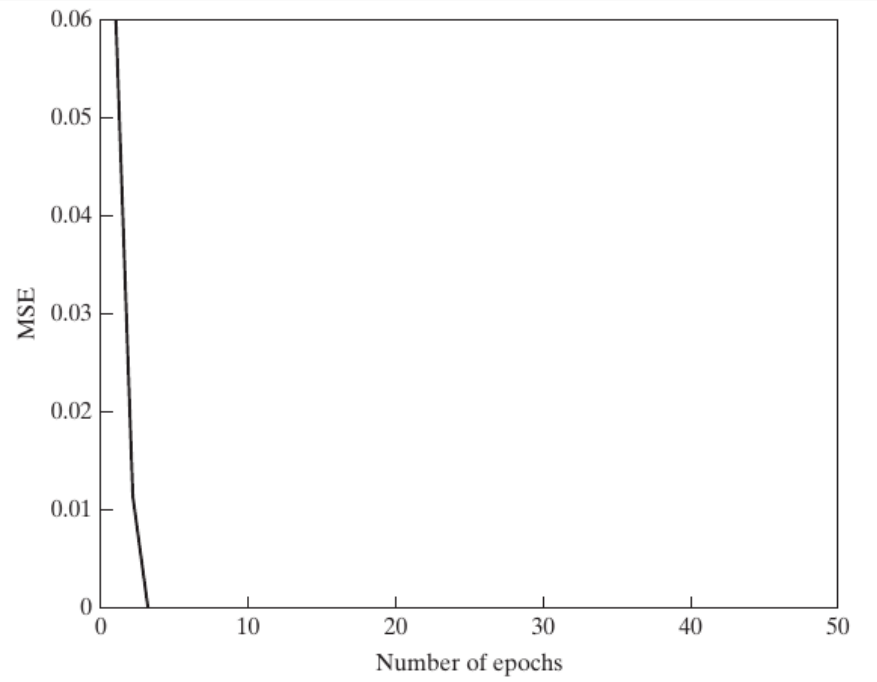
Representativeness

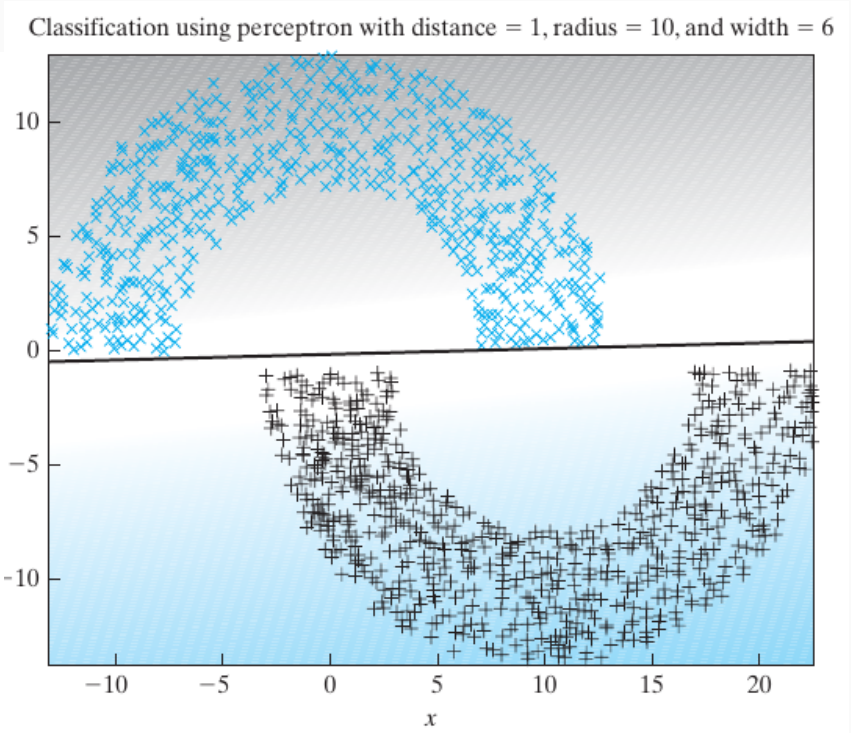


- Represents some useful functions:
- What weights represent $g(x_1, x_2) = AND(x_1, x_2)$?

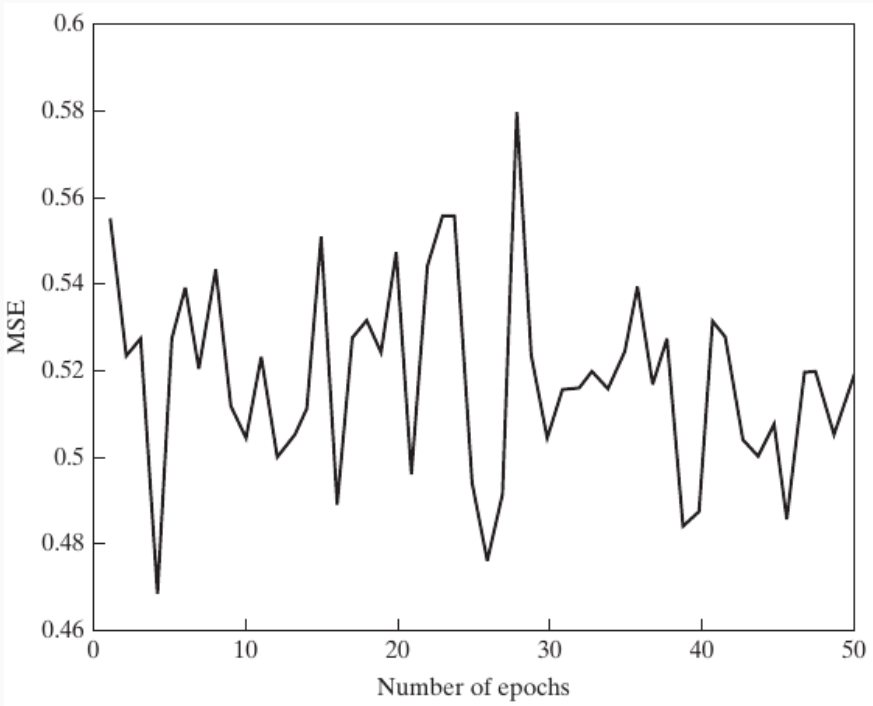
- But some functions not:
- e.g., not linearly separable

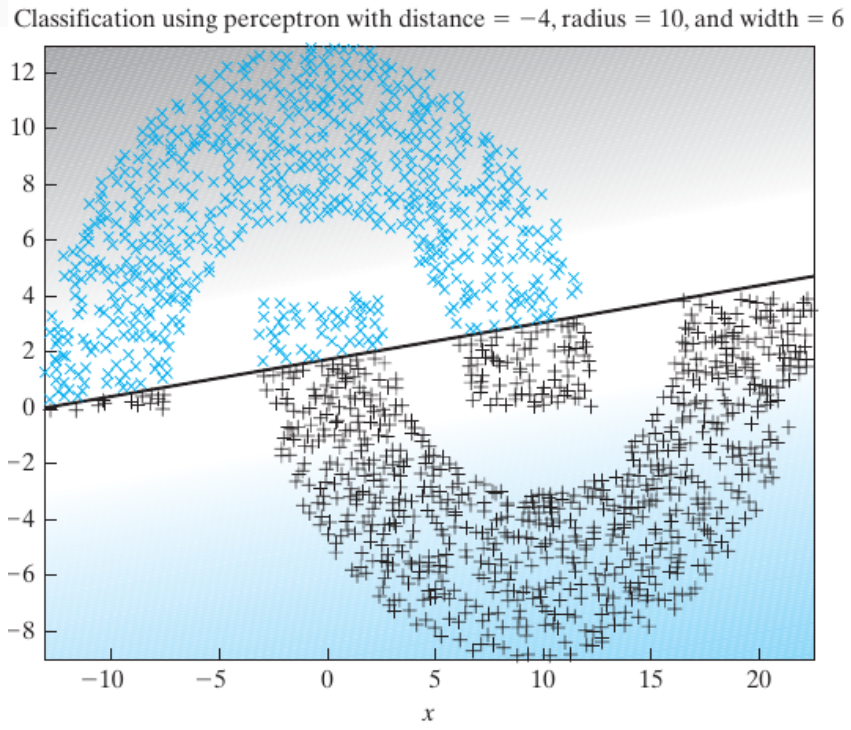
Representativeness: LINEARLY separable





Representativeness: NON-LINEARLY separable





Training rule

$$w_i(new) = w_i + \eta(t - o)x_i \ ,$$

Where:

- $t = c(\vec{x})$ is target value

Perceptron

Training rule

$$w_i(new) = w_i + \eta(t - o)x_i \ ,$$

Where:

- $t = c(\vec{x})$ is target value
- o is perceptron output

Perceptron

Training rule

$$w_i(new) = w_i + \eta(t - o)x_i \ ,$$

Where:

- $t = c(\vec{x})$ is target value
- o is perceptron output
- η is small constant (e.g., 0.1) called *learning rate*

Training rule

$$w_i(new) = w_i + \eta(t - o)x_i \ ,$$

Where:

- $t = c(\vec{x})$ is target value
- o is perceptron output
- η is small constant (e.g., 0.1) called *learning rate*

Can prove it will converge:

- If training data is linearly separable

Training rule

$$w_i(new) = w_i + \eta(t - o)x_i \ ,$$

Where:

- $t = c(\vec{x})$ is target value
- o is perceptron output
- η is small constant (e.g., 0.1) called *learning rate*

Can prove it will converge:

- If training data is linearly separable
- and η sufficiently small

Gradient Descent

- To understand, consider simpler *linear unit*, where:

$$o = w_0 + w_1 x_1 + \cdots + w_n x_n$$

Gradient Descent

- To understand, consider simpler *linear unit*, where:

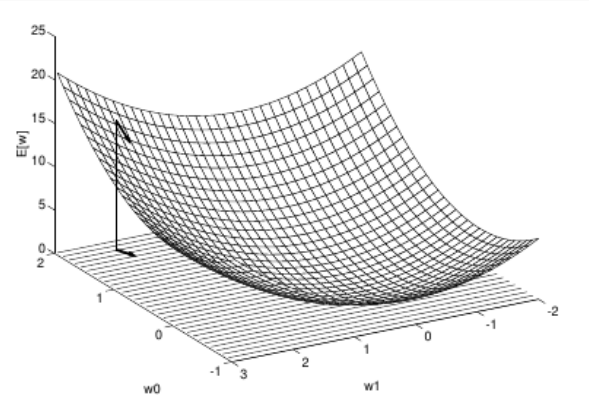
$$o = w_0 + w_1 x_1 + \cdots + w_n x_n$$

- Let's learn w_i's that minimize the squared error:

$$E[\vec{w}] \equiv \frac{1}{2} \sum_{d \in D} (t_d - o_d)^2 \ ,$$

where D is set of training examples

Gradient Descent



- Gradient:

$$\nabla E[\vec{w}] \equiv \left[\frac{\partial E}{\partial w_0}, \frac{\partial E}{\partial w_1}, \cdots \frac{\partial E}{\partial w_n}\right]$$

- Training rule:

$$\Delta \vec{w} = -\eta \nabla E[\vec{w}]$$

- i.e.,

$$w_i(new) = w_i - \eta \frac{\partial E}{\partial w_i}$$

Gradient Descent

$$
\begin{aligned}
\frac{\partial E}{\partial w_i} &= \frac{\partial}{\partial w_i}\frac{1}{2}\sum_d (t_d - o_d)^2 \\
&= \frac{1}{2}\sum_d \frac{\partial}{\partial w_i}(t_d - o_d)^2 \\
&= \frac{1}{2}\sum_d 2(t_d - o_d)\frac{\partial}{\partial w_i}(t_d - o_d) \\
&= \sum_d (t_d - o_d)\frac{\partial}{\partial w_i}(t_d - \vec{w}\cdot\vec{x_d}) \\
\frac{\partial E}{\partial w_i} &= \sum_d (t_d - o_d)(-x_{i,d})
\end{aligned}
$$

Stochastic Gradient Descent

Each training example is a pair of the form $\langle \vec{x}, t \rangle$, where $\vec{x}$ is the vector of input values, and t is the target output value. η is the learning rate (e.g., 0.05).

Stochastic Gradient Descent

Each training example is a pair of the form $\langle \vec{x}, t \rangle$, where $\vec{x}$ is the vector of input values, and t is the target output value. η is the learning rate (e.g., 0.05).

- Initialize each w_i to some small random value

Stochastic Gradient Descent

Each training example is a pair of the form $\langle \vec{x}, t \rangle$, where $\vec{x}$ is the vector of input values, and t is the target output value. η is the learning rate (e.g., 0.05).

- Initialize each w_i to some small random value
- Until the termination condition is met, Do

Stochastic Gradient Descent

Each training example is a pair of the form $\langle \vec{x}, t \rangle$, where $\vec{x}$ is the vector of input values, and t is the target output value. η is the learning rate (e.g., 0.05).

- Initialize each w_i to some small random value
- Until the termination condition is met, Do
  -
  - For each $\langle \vec{x}, t \rangle$ in *training_examples*, Do

Stochastic Gradient Descent

Each training example is a pair of the form $\langle \vec{x}, t \rangle$, where $\vec{x}$ is the vector of input values, and t is the target output value. η is the learning rate (e.g., 0.05).

- Initialize each w_i to some small random value
- Until the termination condition is met, Do
  -
  - For each $\langle \vec{x}, t \rangle$ in *training_examples*, Do
    - Input the instance $\vec{x}$ to the unit and compute the output o

Stochastic Gradient Descent

Each training example is a pair of the form $\langle \vec{x}, t \rangle$, where $\vec{x}$ is the vector of input values, and t is the target output value. η is the learning rate (e.g., 0.05).

- Initialize each w_i to some small random value
- Until the termination condition is met, Do
  -
  - For each $\langle \vec{x}, t \rangle$ in *training_examples*, Do
    - Input the instance $\vec{x}$ to the unit and compute the output o
    - For each linear unit weight w_i, Do: $w_i \leftarrow w_i + \eta(t - o)x_i$

Stochastic Gradient Descent

Each training example is a pair of the form $\langle \vec{x}, t \rangle$, where $\vec{x}$ is the vector of input values, and t is the target output value. η is the learning rate (e.g., 0.05).

- Initialize each w_i to some small random value
- Until the termination condition is met, Do
  -
  - For each $\langle \vec{x}, t \rangle$ in *training_examples*, Do
    - Input the instance $\vec{x}$ to the unit and compute the output o
    - For each linear unit weight w_i, Do: $w_i \leftarrow w_i + \eta(t - o)x_i$

Summary

Perceptron training rule guaranteed to succeed if:

- Training examples are linearly separable
- Sufficiently small learning rate η

Summary

Perceptron training rule guaranteed to succeed if:

- Training examples are linearly separable
- Sufficiently small learning rate η

Linear unit training rule uses gradient descent:

- Guaranteed to converge to hypothesis with minimum squared error

Summary

Perceptron training rule guaranteed to succeed if:

- Training examples are linearly separable
- Sufficiently small learning rate η

Linear unit training rule uses gradient descent:

- Guaranteed to converge to hypothesis with minimum squared error
- Given sufficiently small learning rate η

Summary

Perceptron training rule guaranteed to succeed if:

- Training examples are linearly separable
- Sufficiently small learning rate η

Linear unit training rule uses gradient descent:

- Guaranteed to converge to hypothesis with minimum squared error
- Given sufficiently small learning rate η
- Even when training data contains noise

Summary

Perceptron training rule guaranteed to succeed if:

- Training examples are linearly separable
- Sufficiently small learning rate η

Linear unit training rule uses gradient descent:

- Guaranteed to converge to hypothesis with minimum squared error
- Given sufficiently small learning rate η
- Even when training data contains noise
- Even when training data not separable by H

Outline

Introduction

Neuron

Perceptron

Multi-layer Perceptron

Inductive Bias and Final Remarks

The basics

- The network is composed by the input layer, one or more hidden layers and an output layer of computation nodes.

The basics

- The network is composed by the input layer, one or more hidden layers and an output layer of computation nodes.
- The main novelty regarding the Perceptron is the back-propagation algorithm

The basics

- The network is composed by the input layer, one or more hidden layers and an output layer of computation nodes.
- The main novelty regarding the Perceptron is the back-propagation algorithm
- Back-propagation is based on the error-correction learning rule

The basics

- The network is composed by the input layer, one or more hidden layers and an output layer of computation nodes.
- The main novelty regarding the Perceptron is the back-propagation algorithm
- Back-propagation is based on the error-correction learning rule
- It may be viewed as a generalization of LMS and consists of two steps:
    - Forward pass: producing output(s) as the response of network.

The basics

- The network is composed by the input layer, one or more hidden layers and an output layer of computation nodes.
- The main novelty regarding the Perceptron is the back-propagation algorithm
- Back-propagation is based on the error-correction learning rule
- It may be viewed as a generalization of LMS and consists of two steps:
  - Forward pass: producing output(s) as the response of network.
  - Backward pass: The weights are all adjusted by error-correction rule.

The basics

- The model of each neuron in the network includes a nonlinear activation function.

The basics

- The model of each neuron in the network includes a nonlinear activation function.
- The nonlinearity is smooth and differentiable everywhere by using commonly the sigmoidal nonlinearity.

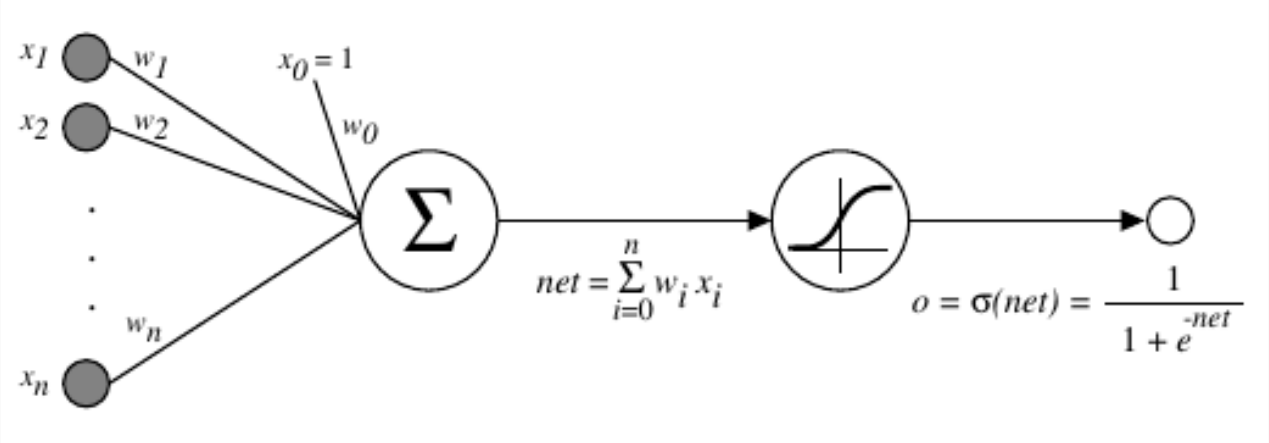
The basics

- The model of each neuron in the network includes a nonlinear activation function.
- The nonlinearity is smooth and differentiable everywhere by using commonly the sigmoidal nonlinearity.
- The network contains one or more layers of hidden neurons which enable the network to learn complex tasks by extracting more meaningful features from the input patterns.

The basics

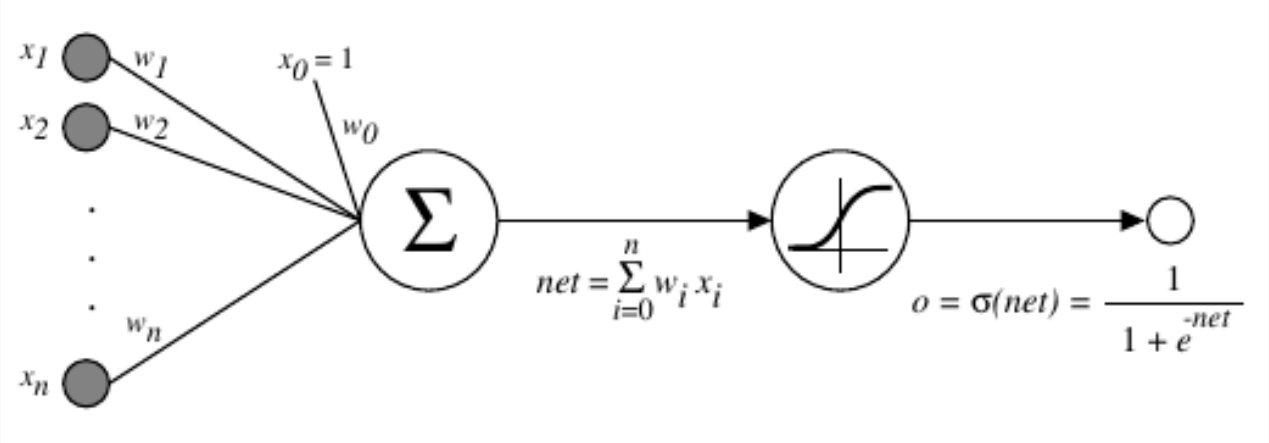
- The model of each neuron in the network includes a nonlinear activation function.
- The nonlinearity is smooth and differentiable everywhere by using commonly the sigmoidal nonlinearity.
- The network contains one or more layers of hidden neurons which enable the network to learn complex tasks by extracting more meaningful features from the input patterns.
- The network exhibits a high degree of connectivity, determined by the synapses of the network.

Sigmoid unit



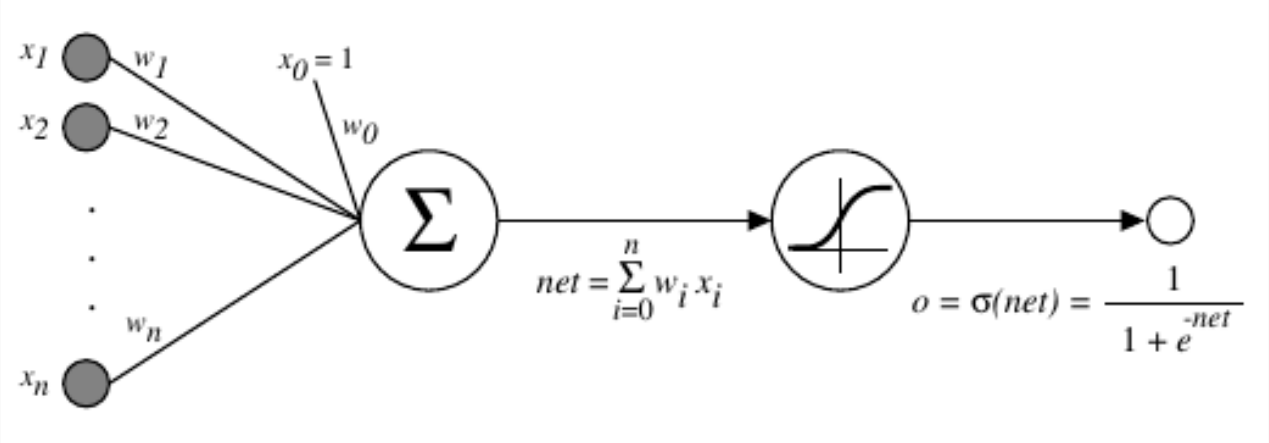
- $\sigma(x)$ is the sigmoid function: $\frac{1}{1+e^{-x}}$

Sigmoid unit



- $\sigma(x)$ is the sigmoid function: $\frac{1}{1+e^{-x}}$
- Nice property: $\frac{d\sigma(x)}{dx} = \sigma(x)(1 - \sigma(x))$

Sigmoid unit



- $\sigma(x)$ is the sigmoid function: $\frac{1}{1+e^{-x}}$
- Nice property: $\frac{d\sigma(x)}{dx} = \sigma(x)(1 - \sigma(x))$
- We can derive gradient descent rules to train:
  - One sigmoid unit
  - *Multilayer networks* of sigmoid units $\rightarrow$ Backpropagation

Error gradient for sigmoid function (part 1)

$$
\begin{aligned}
\frac{\partial E}{\partial w_i} &= \frac{\partial}{\partial w_i} \frac{1}{2} \sum_{d \in D} (t_d - o_d)^2 \\
&= \frac{1}{2} \sum_d \frac{\partial}{\partial w_i} (t_d - o_d)^2 \\
&= \frac{1}{2} \sum_d 2(t_d - o_d) \frac{\partial}{\partial w_i} (t_d - o_d) \\
&= \sum_d (t_d - o_d) \left(-\frac{\partial o_d}{\partial w_i} \right) \\
&= -\sum_d (t_d - o_d) \frac{\partial o_d}{\partial net_d} \frac{\partial net_d}{\partial w_i}
\end{aligned}
$$

Error gradient for sigmoid function (part 2)

$$\frac{\partial E}{\partial w_i} = -\sum_d (t_d - o_d) \frac{\partial \mathbf{o_d}}{\partial \mathbf{net_d}} \frac{\partial \mathbf{net_d}}{\partial \mathbf{w_i}}$$

But we know:

$$\frac{\partial \mathbf{o_d}}{\partial \mathbf{net_d}} = \frac{\partial \sigma(net_d)}{\partial net_d} = o_d(1 - o_d)$$

$$\frac{\partial \mathbf{net_d}}{\partial \mathbf{w_i}} = \frac{\partial(\vec{w} \cdot \vec{x}_d)}{\partial w_i} = x_{i,d}$$

So:

$$\frac{\partial E}{\partial w_i} = -\sum_{d \in D} (t_d - o_d) o_d (1 - o_d) x_{i,d}$$

Back-propagation algorithm

Initialize all weights to small random numbers.

Until satisfied, Do

- For each training example, Do
  1. Input the training example to the network and compute its outputs
  2. For each output unit k

$$\delta_k \leftarrow o_k(1 - o_k)(t_k - o_k)$$

  3. For each hidden unit h

$$\delta_h \leftarrow o_h(1 - o_h) \sum_{k \in outputs} w_{h,k}\delta_k$$

  4. Update each network weight $w_{i,j}$

$$w_{i,j} \leftarrow w_{i,j}(old) + \eta\delta_j x_{i,j}$$

Back-propagation algorithm

- Gradient descent over entire *network* weight vector

Back-propagation algorithm

- Gradient descent over entire *network* weight vector
- Easily generalized to arbitrary directed graphs

Back-propagation algorithm

- Gradient descent over entire *network* weight vector
- Easily generalized to arbitrary directed graphs
- Will find a local, not necessarily global error minimum
    - In practice, often works well (can run multiple times)

Back-propagation algorithm

- Gradient descent over entire *network* weight vector
- Easily generalized to arbitrary directed graphs
- Will find a local, not necessarily global error minimum
  - In practice, often works well (can run multiple times)
- Minimizes error over *training* examples
  - Will it generalize well to subsequent examples?

Back-propagation algorithm

- Gradient descent over entire *network* weight vector
- Easily generalized to arbitrary directed graphs
- Will find a local, not necessarily global error minimum
  - In practice, often works well (can run multiple times)
- Minimizes error over *training* examples
  - Will it generalize well to subsequent examples?
- Training can take thousands of iterations $\rightarrow$ slow!

Back-propagation algorithm

- Gradient descent over entire *network* weight vector
- Easily generalized to arbitrary directed graphs
- Will find a local, not necessarily global error minimum
  - In practice, often works well (can run multiple times)
- Minimizes error over *training* examples
  - Will it generalize well to subsequent examples?
- Training can take thousands of iterations $\rightarrow$ slow!
- Using network after training is very fast

Back-propagation algorithm

- Gradient descent to some local minimum

Back-propagation algorithm

- Gradient descent to some local minimum
  - Perhaps not global minimum...
  - Stochastic gradient descent
  - Train multiple nets with different inital weights

Back-propagation algorithm

- Gradient descent to some local minimum
  - Perhaps not global minimum...
  - Stochastic gradient descent
  - Train multiple nets with different inital weights
- Nature of convergence
  - Initialize weights near zero
  - Therefore, initial networks near-linear
  - Increasingly non-linear functions possible as training progresses

Back-propagation algorithm

- Boolean functions:
  - Every boolean function can be represented by network with single hidden layer
  - but might require exponential (in number of inputs) hidden units
- Continuous functions:
  - Every bounded continuous function can be approximated with arbitrarily small error, by network with one hidden layer
  - Any function can be approximated to arbitrary accuracy by a network with two hidden layers

Outline

Introduction

Neuron

Perceptron

Multi-layer Perceptron

Inductive Bias and Final Remarks

Hypothesis Space Search

Back-propagation

- Every possible assignment of network weights represents a syntactically distinct hypothesis that in principle can be considered by the learner

Hypothesis Space Search

Back-propagation

- Every possible assignment of network weights represents a syntactically distinct hypothesis that in principle can be considered by the learner
- The hypothesis space is the n-dimensional Euclidean space of the n network weights

Hypothesis Space Search

Back-propagation

- Every possible assignment of network weights represents a syntactically distinct hypothesis that in principle can be considered by the learner
- The hypothesis space is the n-dimensional Euclidean space of the n network weights
- Hypothesis space is continuous, in contrast to the hypothesis spaces of decision tree learning and other methods based on discrete representations

Hypothesis Space Search

Back-propagation

- Every possible assignment of network weights represents a syntactically distinct hypothesis that in principle can be considered by the learner
- The hypothesis space is the n-dimensional Euclidean space of the n network weights
- Hypothesis space is continuous, in contrast to the hypothesis spaces of decision tree learning and other methods based on discrete representations
- The fact that it is continuous, together with the fact that E is differentiable with respect to the continuous parameters of the hypothesis, results in a well-defined error gradient that provides a very useful structure for organizing the search for the best hypothesis

Inductive bias

Back-propagation

- What is the inductive bias by which it generalizes beyond the observed data?

Inductive bias

Back-propagation

- What is the inductive bias by which it generalizes beyond the observed data?
- It depends on the interplay between the gradient descent search and the way in which the weight space spans the space of representable functions

Inductive bias

Back-propagation

- What is the inductive bias by which it generalizes beyond the observed data?
- It depends on the interplay between the gradient descent search and the way in which the weight space spans the space of representable functions
- one can roughly characterize it as smooth in- terpolation between data points

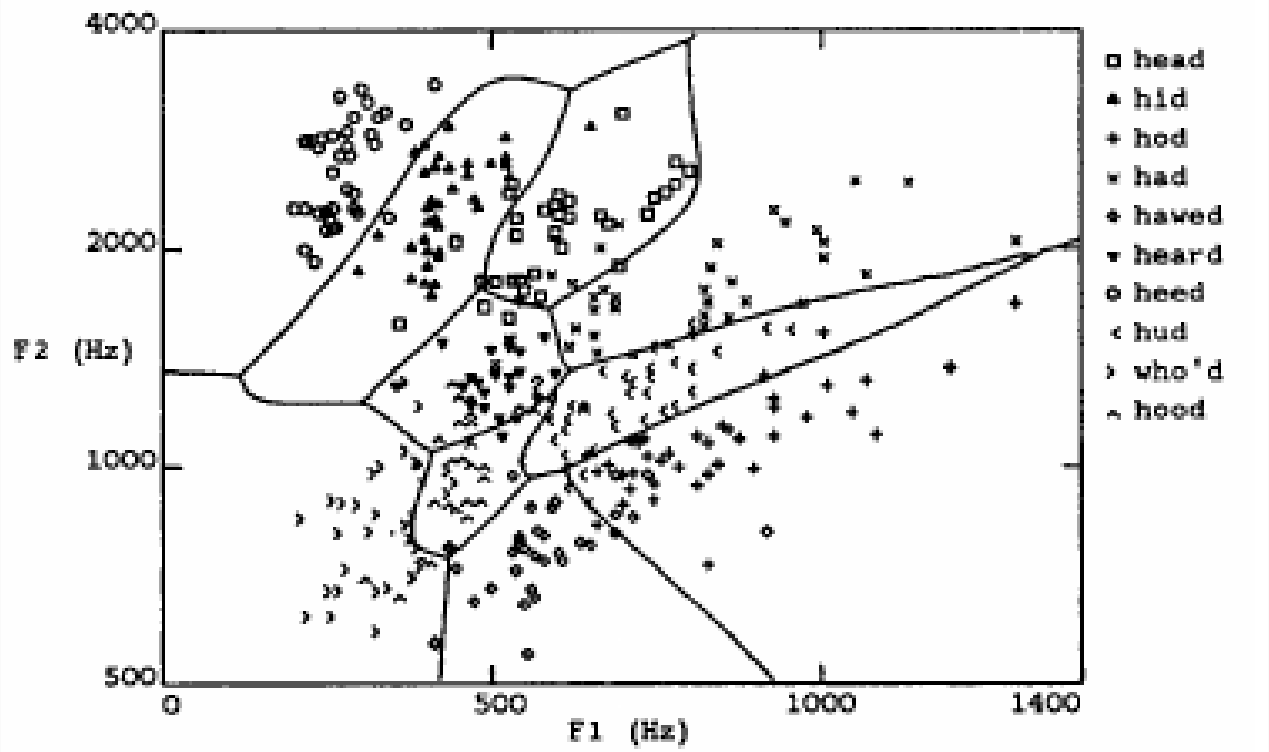
Inductive bias

Back-propagation

- What is the inductive bias by which it generalizes beyond the observed data?
- It depends on the interplay between the gradient descent search and the way in which the weight space spans the space of representable functions
- one can roughly characterize it as smooth in- terpolation between data points
- Given two positive training examples with no negative examples between them, the Back-propagation will tend to label points in between as positive examples as well

Inductive bias

Back-propagation: smooth interpolation between data points



Final Remarks

- What is an appropriate condition for terminating the weight update loop?

Final Remarks

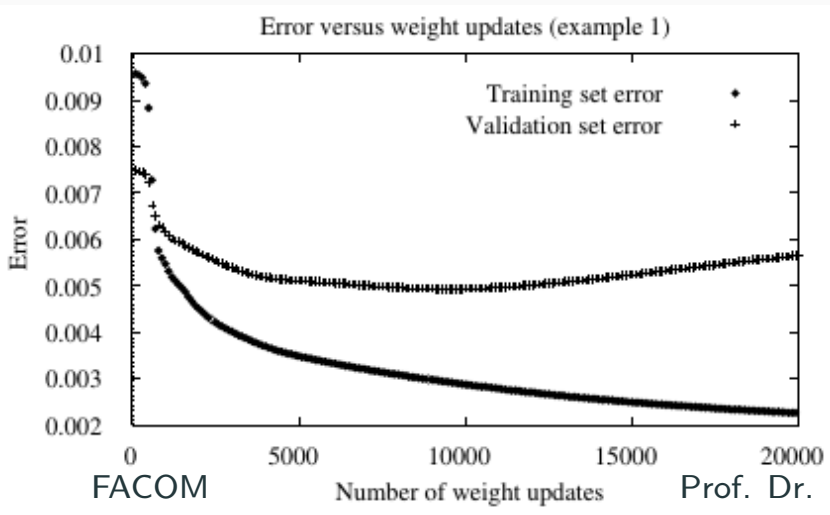
- What is an appropriate condition for terminating the weight update loop?
- Until the error E on the training examples falls below some predetermined threshold?

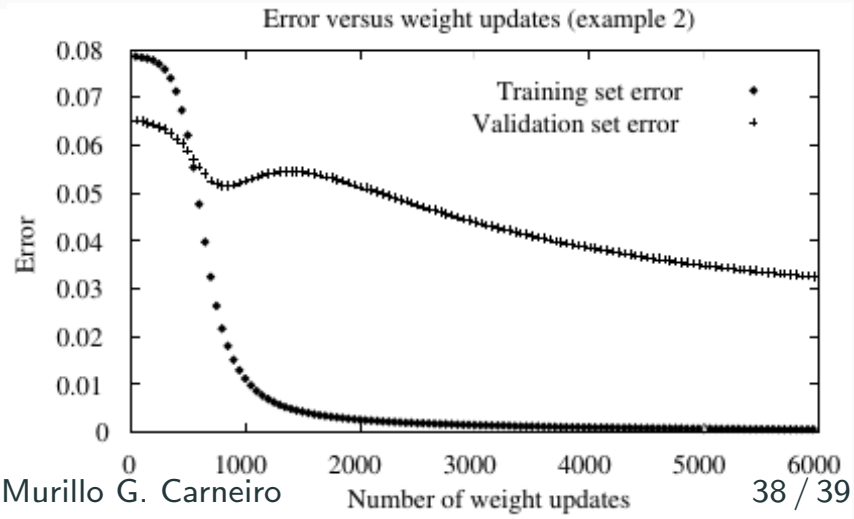
Final Remarks

- What is an appropriate condition for terminating the weight update loop?
- Until the error E on the training examples falls below some predetermined threshold?
- **Be careful about overfitting.** A good recommendation is to monitor the error using a validation set.

Final Remarks

- What is an appropriate condition for terminating the weight update loop?
- Until the error E on the training examples falls below some predetermined threshold?
- **Be careful about overfitting.** A good recommendation is to monitor the error using a validation set.
- Cross-validation can also help too





Referências

- S. S. Haykin. *Neural networks and learning machines*. Pearson, 2009
- T. M. Mitchell. *Machine Learning*. McGraw-Hill, 1997